

ITCS333: Web Development Project Specification

Project Overview

The objective of this project is to design and develop a full-stack, database-driven web application for a **Community Blog Platform**. Users must be able to register, log in, manage their personal profiles, create text-based posts with image uploads, browse a global feed, and securely manage their own content.

Functional Requirements & Page Layout

Your web application must implement a secure authentication system and consist of the following pages/views:

- **Registration Page:** Allows new users to create an account using a unique email address, their full name, and a secure strong password. Proper input validation should be handled.
 - **Login Page:** Authenticates existing users. Upon a successful login, the user must be securely redirected to the Home Page.
 - **Home Page (Global Feed):** Displays all blog posts created by all users across the platform with their pictures (most recent first should be displayed first). Ensure to have a proper navigation menu to access all the other pages, and a welcome message for the logged in user.
 - **Search Page:** Features a search bar allowing users to filter and find posts matching specific keywords or phrases within the post text.
 - **Create Post Page:** Provides a form where authenticated users can write a new post and upload a picture with it (acceptable format .PNG , .JPG).
 - **Post Detail View:** A dedicated page that displays the full details of a single post (author, timestamp, text, and an image) when a user clicks on a post preview from the Home or Search pages.
 - **Post Management (Delete Functionality):** Authenticated users must have the authorization to delete their own posts. A delete option should be accessible either from the Post Detail view or the Home page, ensuring users cannot delete posts belonging to others.
 - **Edit Profile Page:** A dedicated settings page for authenticated users to manage their account details. This page must include options allowing the user to update their **Full Name** and securely change their **Password** (requiring appropriate validation).
-

Database Schema

The system must utilize a relational database structure to handle user records and content management. Below is the required schema blueprint:

```
-- 1. Users Table
-- Stores user credentials and profile details.
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL, -- Stores encrypted passwords for security
  full_name VARCHAR(100) NOT NULL,
```

```
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- 2. Posts Table
-- Stores the blog posts created by users.
CREATE TABLE posts (
    post_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL, -- Foreign key linking the post to its creator
    post_text TEXT NOT NULL,
    image_path VARCHAR(255), -- Stores the file pathway/URL of the uploaded
picture
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
```

UI/UX & Technical Constraints

- **User Interface:** The website must feature a responsive, modern, and user-friendly design.
- **Styling Frameworks:** To achieve a polished look, you are required to use either a pre-made free HTML/CSS web template or a modern CSS framework (such as **Bootstrap**, Tailwind CSS, or Bulma). Raw, unstyled HTML will not be accepted.
- **Session Management & Security:** Pages containing user-specific actions (like creating posts, deleting posts, or changing profile details) must be protected by an authentication mechanism using cookies and only accessible to logged-in users. Implementations may allow the user to remain logged in after closing and reopening their browser/device. Server-side validation must ensure users can only modify or delete their own data.

Javascript usage

Ensure to use Javascript in your project in any creative way. You need to do only one from these.

Examples:

- **Client-Side Form Validation (Registration/Login/Profile):** Before submitting the form. For example, showing a warning immediately if the passwords don't match or if the email format is invalid. (0.5pt)
- **Dynamic Image Preview (Create Post):** When a user selects a picture to upload, JavaScript instantly displays a thumbnail preview on the page before they click submit. (1pt)
- **Confirmation Modals (Delete Post):** Using a dynamic JavaScript popup/modal to ask, "Are you sure you want to delete this post?" to prevent accidental deletions. (0.5pt)
- **Character Counter (Create Post):** A live counter that shows how many characters the user has typed (e.g., "120 / 280 characters remaining") and disables the submit button if they go over the limit. (0.5pt)
- **Theme Toggler (Dark/Light Mode):** A simple button that uses JavaScript to switch the website between light and dark modes, saving the user's preference in localStorage. (0.5pt)

- Asynchronous Search (AJAX/Fetch API): Allow users to search for posts on the Search page and see live results update instantly without reloading the whole page. (1pt)